

Challenge Name: photo memories - 100 (FORENSICS)

Challenge Info

photo memories

100

DescriptionFiles160 Solves

Description

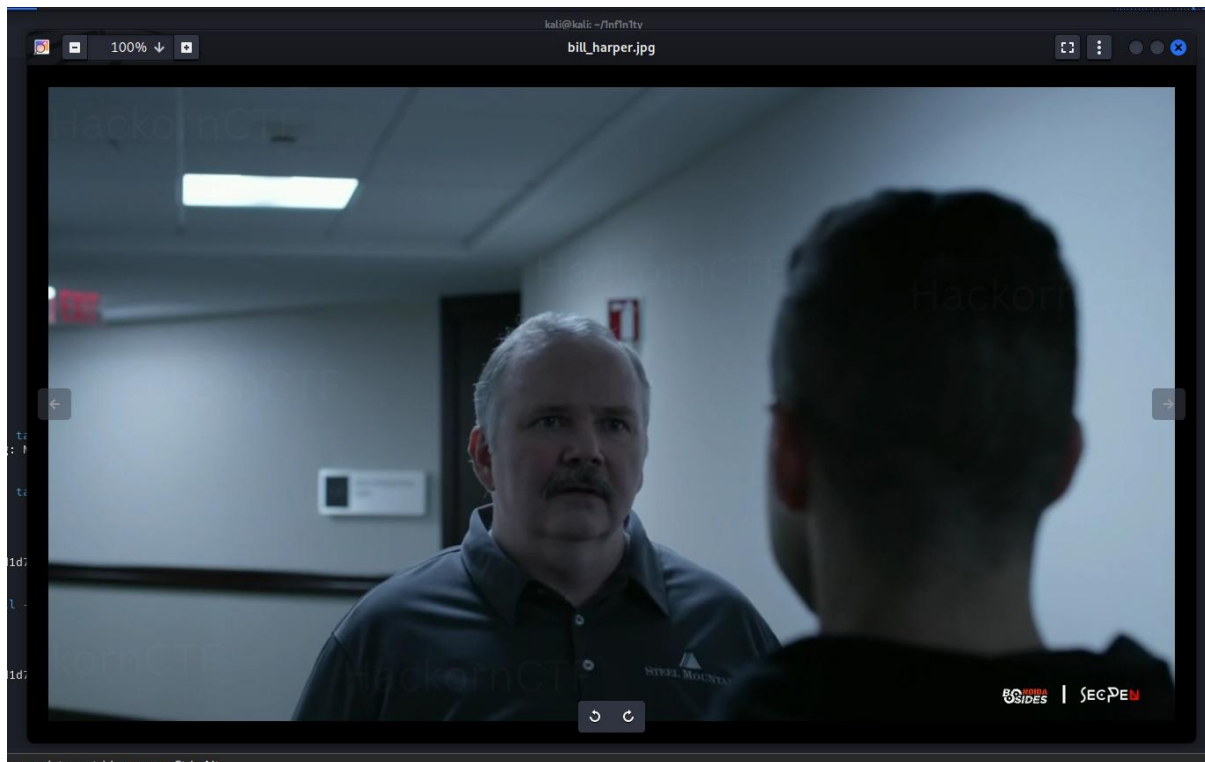
Appearances can lie. @localh0ste

Files

bill_harper.jpg

Flag

The given image is this:



use pointer outside or press Ctrl+Alt

What I did (step-by-step)

1. Download the file

- Using wget, I downloaded in my kali.

```
[kali@kali:~]# ./infinity
[*] http://hackerconrf.scsecn.org/files/b8b27cb9de8284a40d4d6f3ab0626af/bill_harper.jpg?token=eyJ3Y2VxX2l1bnQNdU5lbnRlYmF1bmQ1bG9mZWQ1bG9mLmN0LmE4eWZ7a2Q3PS4eZmVh86v4Y8Y1' -O bill_harper.jpg
--2025-09-26 13:58:01-- http://hackerconrf.scsecn.org/files/b8b27cb9de8284a40d4d6f3ab0626af/bill_harper.jpg?token=eyJ3Y2VxX2l1bnQNdU5lbnRlYmF1bmQ1bG9mZWQ1bG9mLmN0LmE4eWZ7a2Q3PS4eZmVh86v4Y8Y1
100% |#####| 104.21.64.100:80, 92.67.155.210:80, 4790/3011:6815:40ec...
Connecting to hackerconrf.scsecn.org [104.21.64.100]... connected.
HTTP request sent, awaiting response... 200 OK
Length: 282112 (276K) [image/jpeg]
Saving to 'bill_harper.jpg'

bill_harper.jpg                               100%[#####] 275.50K 263KB/s in 1.0s
2025-09-26 13:58:03 (263 kb/s) - 'bill_harper.jpg' saved [282112/282112]
```

2. Basic file checks

- `file bill_harper.jpg`

```
(kali@kali)-[~/1nf1n1ty]
$ file bill_harper.jpg
bill_harper.jpg: JPEG image data, JFIF standard 1.01, resolution (DPI), density 96x96, segment length 16, baseline, precision 8, 1280x720, components 3
```

- `exiftool bill_harper.jpg`

```
(kali@kali)-[~/1nf1n1ty]
$ exiftool bill_harper.jpg
ExifTool Version Number      : 13.10
File Name                    : bill_harper.jpg
Directory                    : .
File Size                    : 282 kB
File Modification Date/Time   : 2025:09:12 03:01:46-04:00
File Access Date/Time        : 2025:09:26 13:58:03-04:00
File Inode Change Date/Time   : 2025:09:26 13:58:03-04:00
File Permissions              : -rw-rw-r--
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                  : 1.01
Resolution Unit               : inches
X Resolution                  : 96
Y Resolution                  : 96
Image Width                   : 1280
Image Height                  : 720
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling          : YCbCr4:2:0 (2 2)
Image Size                    : 1280x720
Megapixels                    : 0.922
```

3. Quick textual search (look for obvious embedded strings / decoys)

- `strings bill_harper.jpg | tail -n 5`

```
(kali@kali)-[~/1nf1n1ty]
$ strings bill_harper.jpg | tail -n 5
=})XaJ
SOi7t
i+-t]z+v
ZU8
SPL{eeff11779250b3bc2b7ac074f86bd1d7}
```

Output contained noisy text plus a visible-looking (but decoy) flag:

```
SPL{eeff11779250b3bc2b7ac074f86bd1d7}
```

Note: that value was a rabbit-hole/fake flag — useful to note but not the real one.

4. Try steghide extraction

I first tried `steghide extract -sf bill_harper.jpg` and when prompted for a passphrase I pressed **Enter** (empty passphrase). That succeeded:

```
(kali@kali)-[~/1nf1n1ty]
$ steghide extract -sf bill_harper.jpg
Enter passphrase:sub.txt
wrote extracted data to "secret.txt".
```

5. Read the extracted file

```
(kali@kali)-[~/1nf1n1ty]
$ cat secret.txt
SPL{2dfa898b8659508904d0c321d3139e4885a441442d7b587e938b31fa84e0d678}
```

Flag captured.

Final

Simple and classic — a stego file with an empty passphrase and a planted decoy. Nice catch!

Flag: SPL{2dfa898b8659508904d0c321d3139e4885a441442d7b587e938b31fa84e0d678}

~By: 1nf1n1ty